

Group members: [Add member names and index numbers]

Original project: [Add original GitHub repository URL and original commit/date]

Modified project: [Add modified GitHub repository URL]

Video: [Add YouTube URL, maximum 20 minutes]

1. Executive Summary

HealthApp is a healthcare management application built with Next.js, TypeScript, React, MongoDB, and custom JWT authentication. It supports patients, doctors, pharmacists, and administrators. The system processes sensitive healthcare information including medical records, prescriptions, allergies, blood groups, emergency contacts, and appointment information.

This assessment identified eight distinct non-payment security vulnerabilities in the original implementation. The findings cover broken access control, authentication bypass, weak cryptography, secret management, object-level authorization, sensitive-data exposure, credential disclosure, and security misconfiguration.

Payment-related vulnerabilities are intentionally excluded from this report scope.

2. Technology and Security Scope

- Next.js App Router
- TypeScript and React
- MongoDB database
- Custom JWT authentication using jose
- HTTP-only authentication cookie
- PBKDF2 password hashing
- Role-based patient, doctor, pharmacist, and administrator workflows
- API route handlers under app/api

The assessment used white-box source inspection. Black-box confirmation using OWASP ZAP and regression tests should be added before final submission.

3. Vulnerability Summary

ID	Vulnerability	Severity	OWASP category
----	---------------	----------	----------------

VULN-01	Privilege escalation during registration	Critical	A01 Broken Access Control
---------	--	----------	---------------------------

VULN-02	Middleware authentication bypass	Critical	A07 Identification and Authentication Failures
---------	----------------------------------	----------	--

VULN-03	Weak password hashing	High	A02 Cryptographic Failures
---------	-----------------------	------	----------------------------

VULN-04	Hard-coded JWT secret fallback	Critical	A02 Cryptographic Failures
---------	--------------------------------	----------	----------------------------

VULN-05 Broken object-level authorization / IDOR High A01 Broken Access Control
VULN-06 Excessive medical-data exposure High A01 Broken Access Control
VULN-07 Password-hash disclosure High A02 Cryptographic Failures
VULN-08 Security misconfiguration Medium A05 Security Misconfiguration

4. Detailed Findings

VULN-01: Privilege Escalation During Registration

Severity: Critical

Description: The public registration endpoint accepts the role value directly from the request. A user can register as an administrator, doctor, or pharmacist without administrator approval.

Evidence: `app/api/auth/register/route.ts` accepts role from the request body, stores it in the user document, and places it in the JWT.

Impact: An unauthenticated attacker can create a privileged account and access administrative or staff functionality.

Recommended fix: Public registration must always create a patient account. Staff accounts should be created only through an authenticated administrator workflow or invitation process. The server must use an allowlist and must never trust a client-provided privileged role.

Verification evidence: Attempt registration with role: "admin"; the secure version should reject the request or create only a patient account.

VULN-02: Middleware Authentication Bypass

Severity: Critical

Description: The middleware identifies public paths using `pathname.startsWith(path)`. Since `/` is included in `publicPaths`, every pathname starts with `/`. As a result, protected page routes can bypass middleware authentication.

Evidence: `middleware.ts` defines `const publicPaths = ["/auth/login", "/auth/register", "/"]` and checks `pathname.startsWith(path)`.

Impact: Unauthenticated users may directly access protected application pages. API routes still require their own checks, but the page-level security boundary is ineffective.

Recommended fix: Match exact public routes or use a safe route predicate. For example, compare `pathname === "/"` instead of using `startsWith` for the root route. Add middleware tests for anonymous access to every protected role area.

Verification evidence: Request `/admin/dashboard`, `/doctor/dashboard`, and `/patient/dashboard` without a cookie. The secure version should redirect to login.

VULN-03: Weak Password Hashing

Severity: High

Description: Passwords are hashed with PBKDF2-HMAC-SHA512 using only 1,000 iterations. This is substantially weaker than modern password-storage recommendations and makes offline cracking cheaper if hashes are obtained.

Evidence: lib/password.ts calls pbkdf2 with 1000 iterations.

Impact: Compromised password hashes may be cracked more efficiently, especially when users reuse passwords.

Recommended fix: Use a modern password hashing algorithm such as Argon2id or a properly configured bcrypt cost factor. Store an algorithm/version marker so hashes can be upgraded after successful login. Use constant-time comparison and enforce server-side password policy.

Verification evidence: Add tests confirming new passwords use the selected algorithm and that legacy hashes are migrated safely.

VULN-04: Hard-Coded JWT Secret Fallback

Severity: Critical

Description: The JWT implementation uses your-secret-key-change-in-production when JWT_SECRET is missing.

Evidence: lib/auth.ts defines the signing key using process.env.JWT_SECRET "your-secret-key-change-in-production".

Impact: If the environment variable is absent, an attacker who knows the source code can forge tokens and impersonate any role, including administrators.

Recommended fix: Fail application startup when JWT_SECRET is missing or too short. Use a high-entropy deployment secret, rotate secrets safely, and avoid embedding fallback credentials in source code.

Verification evidence: Start the application without JWT_SECRET; the secure version should fail configuration validation rather than start with a default key.

VULN-05: Broken Object-Level Authorization / IDOR

Severity: High

Description: Doctor-only creation endpoints accept arbitrary patient identifiers. They check the caller's role but do not verify that the doctor is authorized to access the selected patient.

Evidence: app/api/medical-records/create/route.ts and the prescription creation route accept patientId without checking a valid doctor-patient appointment, hospital relationship, or treatment relationship.

Impact: A doctor account could create or manipulate records belonging to unrelated patients.

This violates confidentiality and integrity requirements for healthcare data.

Recommended fix: Centralize authorization checks. Confirm that the patient exists and that the doctor has a valid appointment or approved clinical relationship. Apply the same checks to read, create, update, and delete operations.

Verification evidence: Authenticate as Doctor A and submit a record for Patient B without a relationship. The secure version should return 403 Forbidden.

VULN-06: Excessive Medical-Data Exposure

Severity: High

Description: Health-card scanning returns a broad collection of patient information, including demographics, allergies, records, prescriptions, and appointments.

Evidence: `app/api/health-card/scan/route.ts` returns multiple sensitive healthcare datasets after a card lookup.

Impact: A compromised or misused doctor account can access more personal health information than necessary for a specific clinical task.

Recommended fix: Apply data minimization. Return only the fields required for the current workflow, require an authorized relationship or appointment, add consent and audit logging, and separate emergency access from ordinary access.

Verification evidence: Scan a card as an unrelated doctor. The secure version should deny access or return only the minimum permitted fields.

VULN-07: Password-Hash Disclosure

Severity: High

Description: The admin staff endpoint returns staff documents without explicitly excluding the password field. Even a password hash must not be exposed through an API response.

Evidence: `app/api/admin/staff/route.ts` returns staff documents without a password projection.

Impact: A compromised administrator session, frontend bug, logging system, or browser extension could obtain password hashes and enable offline cracking.

Recommended fix: Use explicit response projections or DTOs that allow only safe fields. Never return the password field from any endpoint. Add automated tests that recursively check sensitive fields are absent from API responses.

Verification evidence: Call the staff endpoint as an administrator and confirm that no password property is returned.

VULN-08: Security Misconfiguration

Severity: Medium

Description: The application does not visibly configure important HTTP security headers, and the Next.js configuration ignores TypeScript and ESLint errors during builds.

Evidence: next.config.mjs disables build failure for TypeScript and ESLint errors. No application-wide CSP, HSTS, frame protection, MIME-sniffing protection, or Referrer-Policy configuration was identified.

Impact: Missing headers increase exposure to clickjacking, MIME confusion, information leakage, and script-injection impact. Ignoring build errors can allow security defects to reach deployment.

Recommended fix: Configure security headers in middleware or Next.js headers configuration. Enable TypeScript and lint failures in CI and production builds. Add automated checks for required headers.

Verification evidence: Inspect response headers with OWASP ZAP or browser developer tools and confirm required headers are present in the secure version.

5. Additional Security Observations

The following issues should be assessed and may be included as additional findings after testing:

- JWTs remain valid for seven days after logout because there is no revocation or token-version mechanism.
- QR codes contain patient identifiers and have no expiry, signature, nonce, or revocation mechanism.
- Password-change verification uses bcrypt while login uses PBKDF2, causing password changes to fail for normal PBKDF2 users.
- State-changing cookie-authenticated APIs do not show explicit CSRF tokens or origin validation.
- Several API routes perform minimal validation and have no visible rate limiting, pagination, or request-size policy.
- Dependency vulnerabilities must be confirmed with npm audit before being reported as CVEs.
- SQL injection is not applicable because the application uses MongoDB. XSS and NoSQL injection require black-box confirmation before being claimed.

6. Recommended Testing Plan

1. Create a baseline copy or Git tag of the original vulnerable implementation.
2. Add one security regression test for each vulnerability.
3. Use OWASP ZAP against a local test deployment.
4. Test anonymous access to protected pages.
5. Test role tampering during registration.
6. Test cross-patient medical-record requests.
7. Test sensitive-field leakage from admin endpoints.
8. Inspect security response headers.
9. Run npm audit and record the result separately from source-code findings.

10. Repeat the same tests after remediation and retain screenshots or test output for the report.

7. Secure Software Engineering Practices

These vulnerabilities could have been prevented through:

- Server-side allowlists for roles and permissions.
- Centralized authorization middleware and resource ownership checks.
- Secure secret validation during startup.
- Modern password hashing standards.
- Schema validation for every API payload.
- Data-minimizing response DTOs.
- Threat modeling for healthcare data and QR workflows.
- Security-focused code review and pull-request checks.
- Dependency scanning and lockfile maintenance.
- Automated security regression tests and OWASP ZAP scans in CI.
- Fail-closed production builds that do not ignore type or lint errors.

8. Limitations

This initial report is based on source-code inspection. Database indexes, production deployment headers, environment variables, live MongoDB permissions, and runtime behavior were not independently verified. The final submission should clearly label black-box findings, white-box findings, and assumptions.

9. Group Contributions

- Member 1: [Name, index, contribution]
- Member 2: [Name, index, contribution]
- Member 3: [Name, index, contribution]
- Member 4: [Name, index, contribution]